6.5830/1 Lecture 14: Recovery (ctd)

4/2/2026

Tianyu Li

litianyu@mit.edu

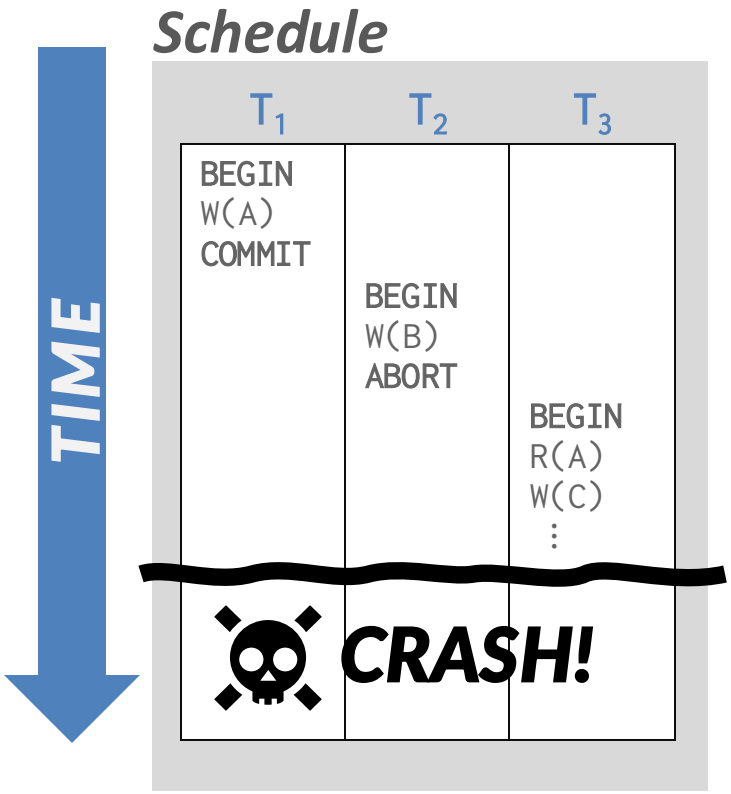
Many thanks to Andy Pavlo and CMU DB for sharing their materials

Administrivia

- Lab 4 will be out later today
  - 6.5831 only
  - Autograder will be released later
  - Due last day of class (05/12) before lecture, no late days

Last Class

- **Database Recovery ensure consistency, atomicity, durability in the face of failures**
- **STEAL + NO-FORCE**
- Desired behavior after the DBMS restarts (i.e., the contents of volatile memory are lost):
  - T_1 should be durable.
  - T_2 + T_3 should be aborted.



ARIES

- **Algorithms for Recovery and Isolation Exploiting Semantics**
- Developed at IBM Research in early 1990s for the DB2 DBMS.
- Not all systems implement ARIES exactly as defined in this paper but they're close enough.

ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging

C. MOHAN
IBM Almaden Research Center
and
DON HADERLE
IBM Santa Teresa Laboratory
and
BRUCE LINDSAY, HAMID PIRAHESH and PETER SCHWARZ
IBM Almaden Research Center

In this paper we present a simple and efficient method, called ARIES (*Algorithm for Recovery and Isolation Exploiting Semantics*), which supports partial rollbacks of transactions, fine-granularity (e.g., record) locking and recovery using write-ahead logging (WAL). We introduce the paradigm of *repeating history* to redo all missing updates *before* performing the rollbacks of the loser transactions during restart after a system failure. ARIES uses a log sequence number in each page to correlate the state of a page with respect to logged updates of that page. All updates of a transaction are logged, including those performed during rollbacks. By appropriate chaining of the log records written during rollbacks to those written during forward progress, a bounded amount of logging is ensured during rollbacks even in the face of repeated failures during restart or of nested rollbacks. We deal with a variety of features that are very important in building and operating an *industrial-strength* transaction processing system. ARIES supports fuzzy checkpoints, selective and deferred restart, fuzzy image copies, media recovery, and high concurrency lock modes (e.g., increment/decrement) which exploit the semantics of the operations and require the ability to perform operation logging. ARIES is flexible with respect to the kinds of buffer management policies that can be implemented. It supports objects of varying length efficiently. By enabling parallelism during restart, page-oriented redo, and logical undo, it enhances concurrency and performance. We show why some of the System R paradigms for logging and recovery, which were based on the shadow page technique, need to be changed in the context of WAL. We compare ARIES to the WAL-based recovery methods of

Authors' addresses: C. Mohan, Data Base Technology Institute, IBM Almaden Research Center, San Jose, CA 95120; D. Haderle, Data Base Technology Institute, IBM Santa Teresa Laboratory, San Jose, CA 95150; B. Lindsay, H. Pirahesh, and P. Schwarz, IBM Almaden Research Center, San Jose, CA 95120.

ACM Transactions on Database Systems, Vol. 17, No. 1, March 1992, Pages 94–162

ARIES Approach: 3 Log Passes

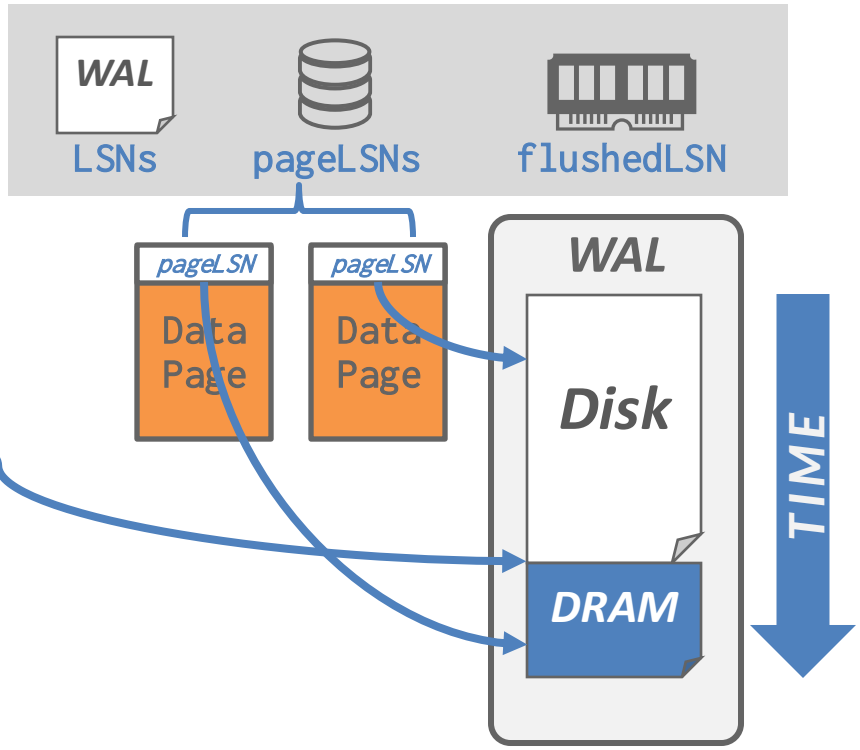
- **Analysis**, to see what needs to be done
- **Redo**, to ensure DB reflects updates that are in the log but not in tables
  - Including those that belong to txns that will eventually be rolled back!
  - Why? Ensures we are at exactly the state before we failed -- which will allow logical undo.
- **Undo**, to rollback uncommitted transactions

WAL Records

- We need to extend our log record format from last class to include additional info.
- Every log record includes a globally unique, monotonicially increasing **log sequence number** (LSN).
  - LSNs represent the physical order that txns make changes to the database.
  - In GoDB, simply the physical offset on log

WAL Bookkeeping

- Each data page contains a pageLSN.
  - The LSN of the most recent log record that updated the page.
- System keeps track of flushedLSN.
  - The max LSN flushed so far.
- WAL: Before a $page_x$ is written, $pageLSN_x$ < flushedLSN



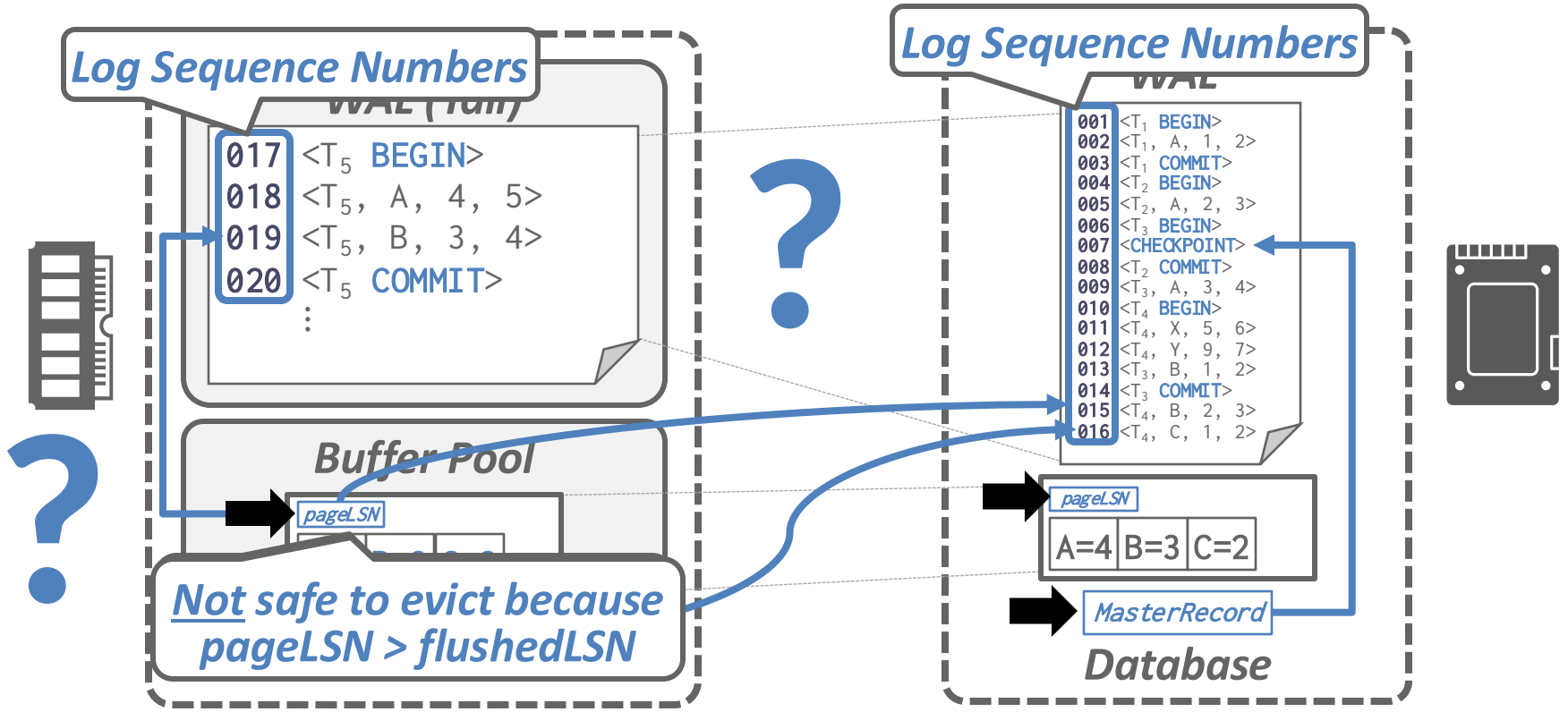
LSN Usage

Name	Location	Definition
flushedLSN	Memory	Last LSN in log on disk
pageLSN	$page_x$	Newest update to $page_x$
recLSN	DPT♦	Oldest update to $page_x$ since it was last flushed
lastLSN	ATT♣	Latest record of txn T_i
MasterRecord	Disk	LSN of latest checkpoint

♦ DPT = Dirty Page Table.

♣ ATT = Active Transaction Table.

Writing Log Records



Writing Log Records

- All log records have an LSN.
- Update the pageLSN every time a txn modifies a record in the page.
  - Remember, log records appended *before* making modifications
- Update the flushedLSN in memory every time the DBMS writes the WAL buffer to disk.

Normal Execution

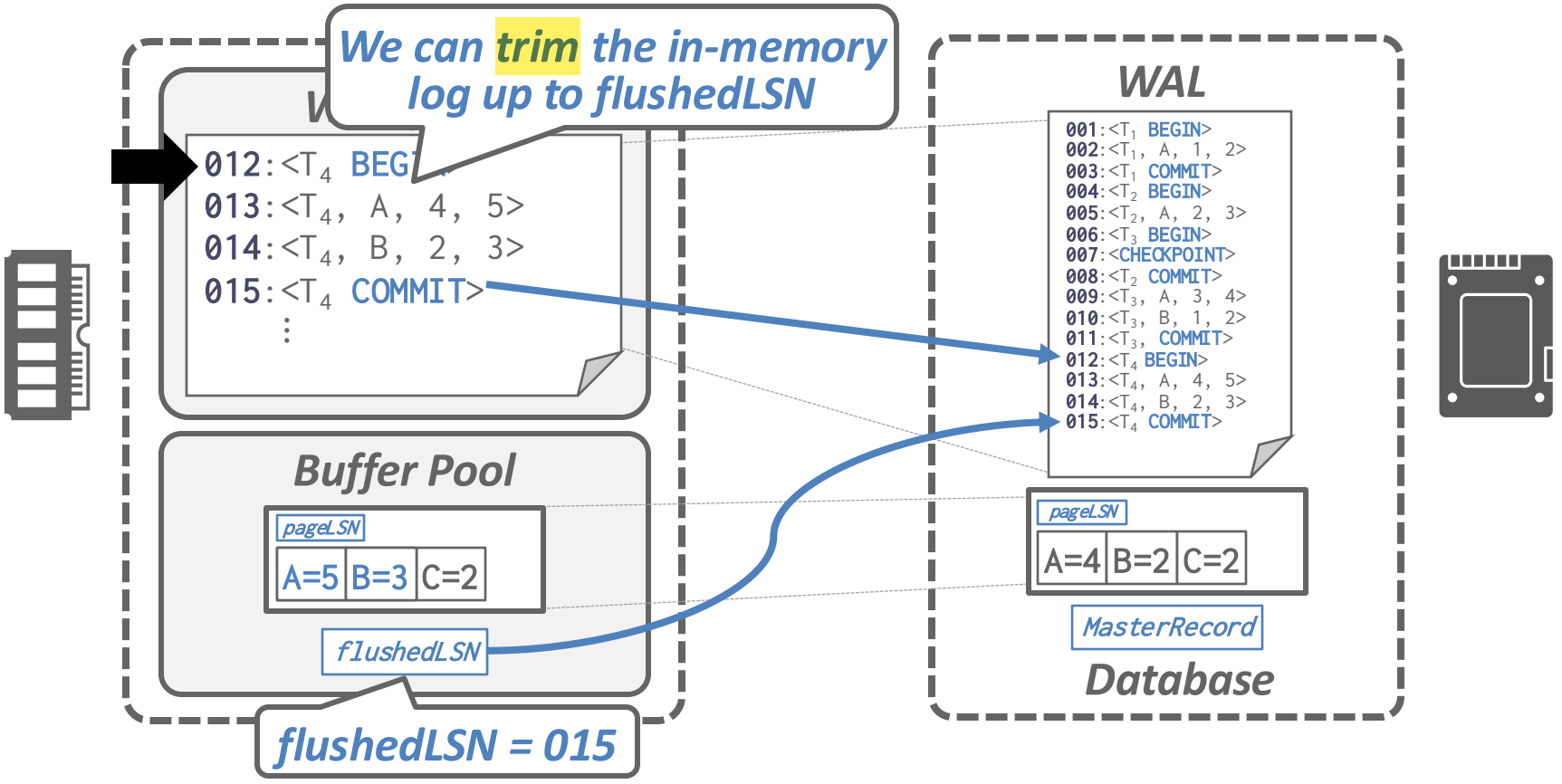
- Each txn invokes a sequence of reads and writes, followed by commit or rollback.
- Assumptions in this lecture:
  - All log records fit within a single page.
  - Disk writes are atomic.
  - Single-versioned tuples with Strong Strict 2PL.
  - **STEAL** + **NO-FORCE** buffer management with WAL.
  - Physical log record scheme.
  - Omitting log records for indexes.

Commit Path

- When a txn commits, the DBMS writes a `COMMIT` record to log
  - Txn is considered committed when this record is on disk
  - Because logs are flushed prefix-at-a-time, all previous changes must also be on disk

- If there is no `COMMIT` record, a transaction is presumed to abort

Commit Path


We can trim the in-memory log up to flushedLSN
012:<T4 BEGIN>
013:<T4, A, 4, 5>
014:<T4, B, 2, 3>
015:<T4 COMMIT>
Buffer Pool
pageLSN
A=5 B=3 C=2
flushedLSN
flushedLSN = 015
WAL
001:<T1 BEGIN>
002:<T1, A, 1, 2>
003:<T1 COMMIT>
004:<T2 BEGIN>
005:<T2, A, 2, 3>
006:<T3 BEGIN>
007:<CHECKPOINT>
008:<T2 COMMIT>
009:<T3, A, 3, 4>
010:<T3, B, 1, 2>
011:<T3, COMMIT>
012:<T4 BEGIN>
013:<T4, A, 4, 5>
014:<T4, B, 2, 3>
015:<T4 COMMIT>
pageLSN
A=4 B=2 C=2
MasterRecord
Database

Abort Path

- Aborting a txn is significant trickier than commit
- We need to add another field to our log records:
  - prevLSN: The previous **LSN** for the txn.
  - This maintains a linked-list for each txn to make it easier to walk through its log records.
  - Note: GoDB uses a different scheme

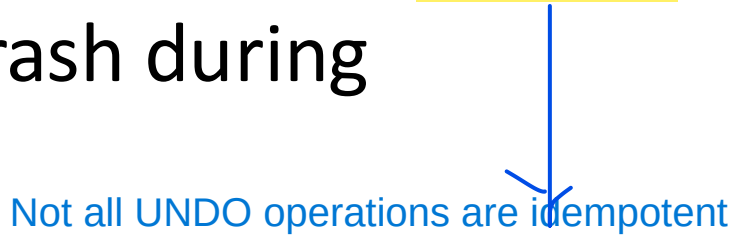
What's wrong?

- Strawman:
  - Apply undo in reverse order
  - Release locks after all changes reverted
  - Write ABORT record to log

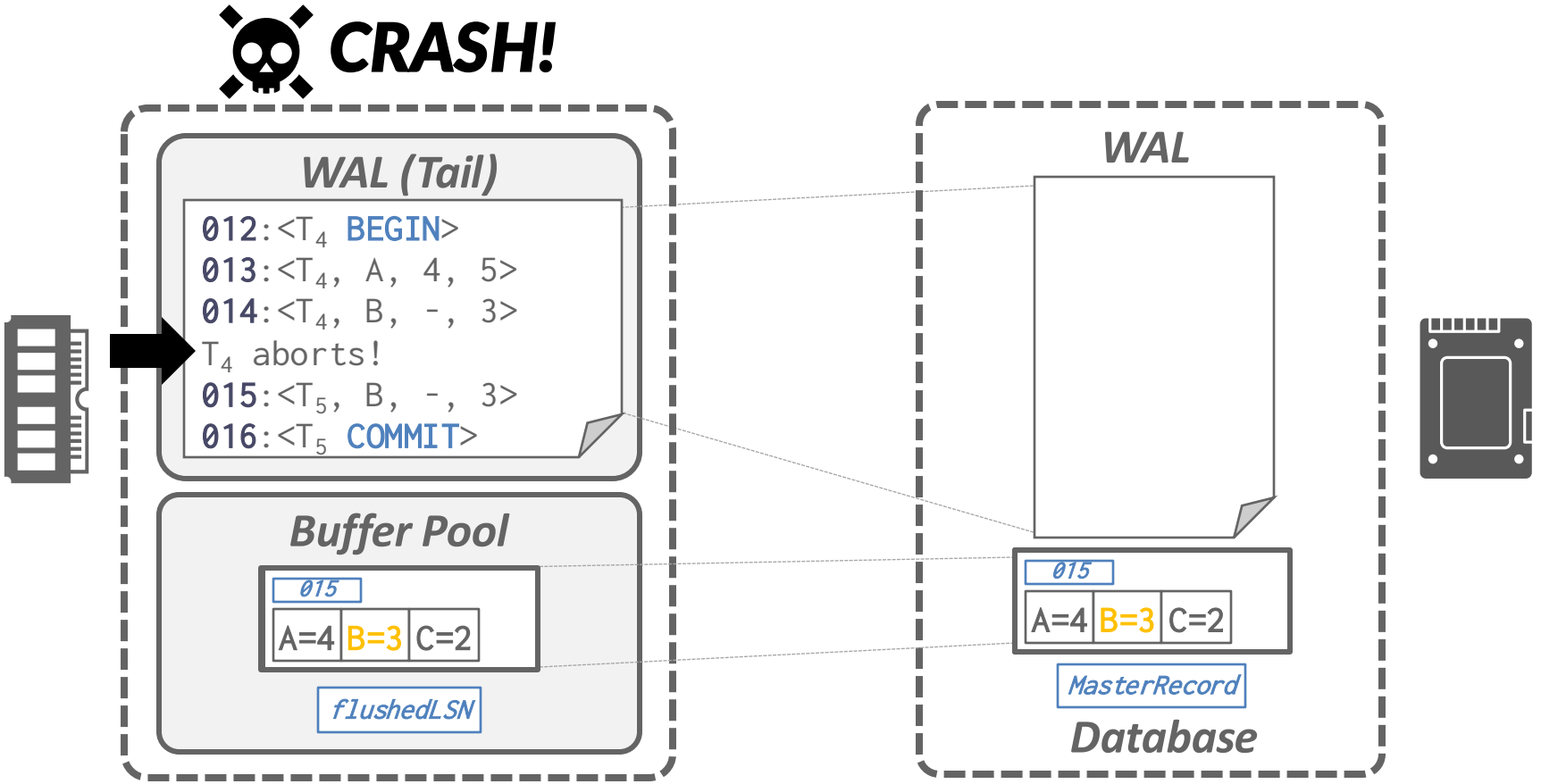
Problem

- Key problem: your database can crash while undoing / recovering
  - Murphy strikes when you least expect him to

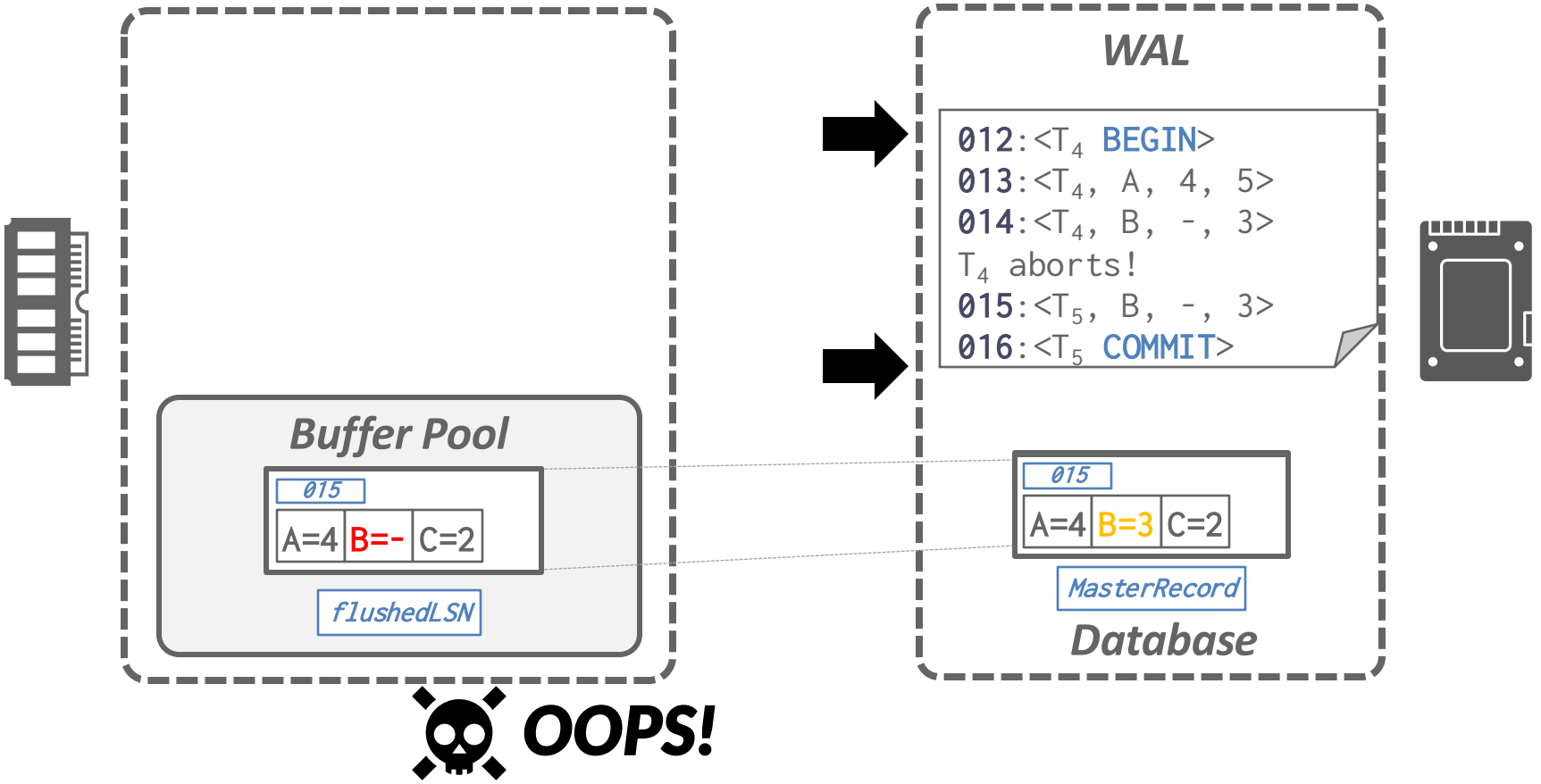
- Important consideration – can we corrupt the database state if we crash during recovery/rollback?



Problem


CRASH!
WAL (Tail)
012:<T4 BEGIN>
013:<T4, A, 4, 5>
014:<T4, B, -, 3>
T4 aborts!
015:<T5, B, -, 3>
016:<T5 COMMIT>
Buffer Pool
015
A=4 B=3 C=2
flushedLSN
WAL
015
A=4 B=3 C=2
MasterRecord
Database

Problem


WAL
012:<T4 BEGIN>
013:<T4, A, 4, 5>
014:<T4, B, -, 3>
T4 aborts!
015:<T5, B, -, 3>
016:<T5 COMMIT>
Buffer Pool
015
A=4 B=- C=2
flushedLSN
015
A=4 B=3 C=2
MasterRecord
Database
OOPS!

Problem

- Takeaway: undos in ARIES are not idempotent
  - Unavoidable result of (physio)logical undo
- Must track undo progress to ensure we only apply it once
- But wait, redos are idempotent
  - Thanks to pageLSN check

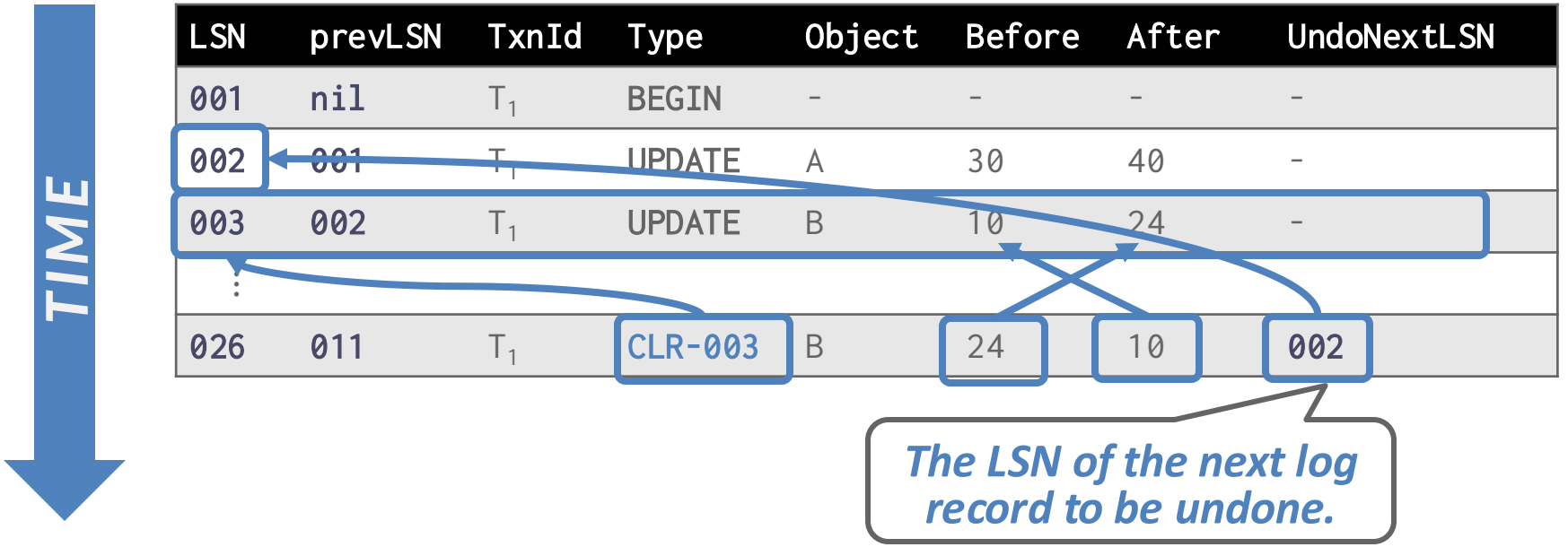
Idea

- Every rollback now generates a redo record
  - With its own unique lsn, logged before changes are applied
  - This redo record "cancels" the previous undo record
- This is called a "compensation log record (CLR)"

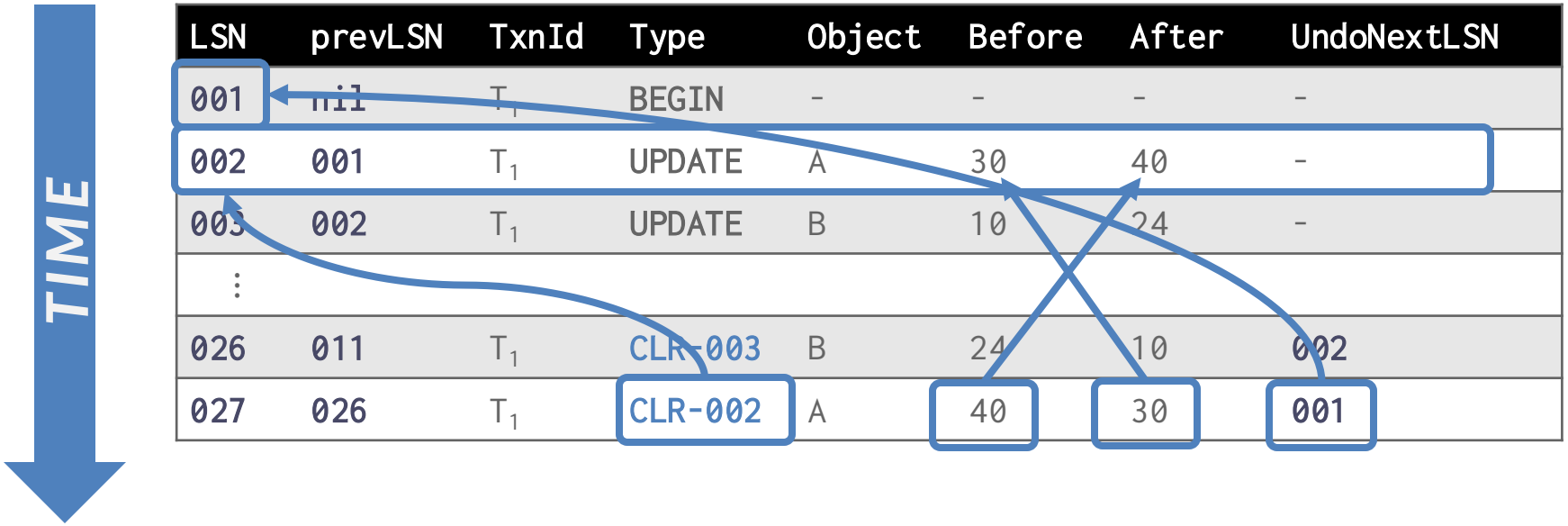
CLR: Example

LSN	prevLSN	TxnId	Type	Object	Before	After	UndoNextLSN
001	nil	T_1	BEGIN	-	-	-	-
002	001	T_1	UPDATE	A	30	40	-
003	002	T_1	UPDATE	B	10	24	-
⋮							

CLR: Example


TIME
LSN
prevLSN
TxnId
Type
Object
Before
After
UndoNextLSN
001
nil
T1
BEGIN
-
-
-
-
002
001
T1
UPDATE
A
30
40
-
003
002
T1
UPDATE
B
10
24
-
026
011
T1
CLR-003
B
24
10
002
The LSN of the next log record to be undone.

CLR: Example


TIME
LSN prevLSN TxnId Type Object Before After UndoNextLSN
001 nil T1 BEGIN - - - -
002 001 T1 UPDATE A 30 40 -
003 002 T1 UPDATE B 10 24 -
026 011 T1 CLR-003 B 24 10 002
027 026 T1 CLR-002 A 40 30 001

CLR: Example

LSN	prevLSN	TxnId	Type	Object	Before	After	UndoNextLSN
001	nil	T_1	BEGIN	-	-	-	-
002	001	T_1	UPDATE	A	30	40	-
003	002	T_1	UPDATE	B	10	24	-
⋮							
026	011	T_1	CLR-003	B	24	10	002
027	026	T_1	CLR-002	A	40	30	001
028	027	T_1	ABORT	-	-	-	nil

Abort Algorithm

- Undo the txn's updates in reverse order. For each update record:
  - Write a CLR entry to the log.
  - Restore old value.

- Lastly, write an ABORT record and release locks.

Refresher: Checkpoints

- Last lecture, we talked about blocking checkpoints:

→Halt the start of any new txns.

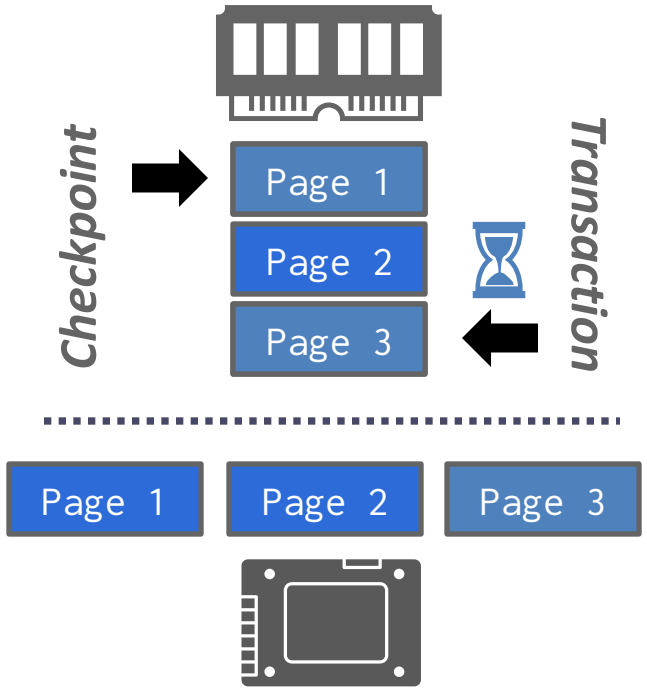
→Wait until all active txns finish executing.

→Flushes dirty pages on disk.

- This checkpoint implementation is bad for runtime performance, but it makes recovery easy.

How to do (slightly) better?

- Pause modifying txns while the DBMS takes the checkpoint.
  - Flush all dirty pages
- Problem: pages contain uncommitted writes that may need to be rolled back
  - Cannot truncate log up until checkpoint
- We must record internal state as of the beginning of the checkpoint.



Active Transactions Table (ATT)

- One entry per currently active txn.
  - `txnId`: Unique txn identifier.
  - `status`: The current status of the txn.
  - `lastLSN`: Most recent LSN created by the txn.

- Remove a txn's ATT entry after appending its end record to the in-memory WAL buffer.

Slightly Better Checkpoints

- At the first checkpoint, assuming P_{11} was flushed, T_2 is still running and there is only one dirty page (P_{22}).
- At the second checkpoint, assuming P_{22} was flushed, T_2 and T_3 are active and the dirty pages are (P_{11}, P_{33}).
- Safe to truncate log records earlier than the earliest of the active transactions
  - Play forward from checkpoint record

```
001:<T1 BEGIN>
002:<T2 BEGIN>
003:<T1, A→P11, 100, 120>
004:<T1 COMMIT>
005:<T2, C→P22, 100, 120>
006:<CHECKPOINT
     ↳ATT={T2}>
007:<T3 BEGIN>
008:<T2, A→P11, 120, 130>
009:<T2 COMMIT>
010:<T3, B→P33, 200, 400>
011:<CHECKPOINT
     ↳ATT={T2,T3}>
012:<T3, B→P33, 400, 600>
```

Much Better Checkpoints

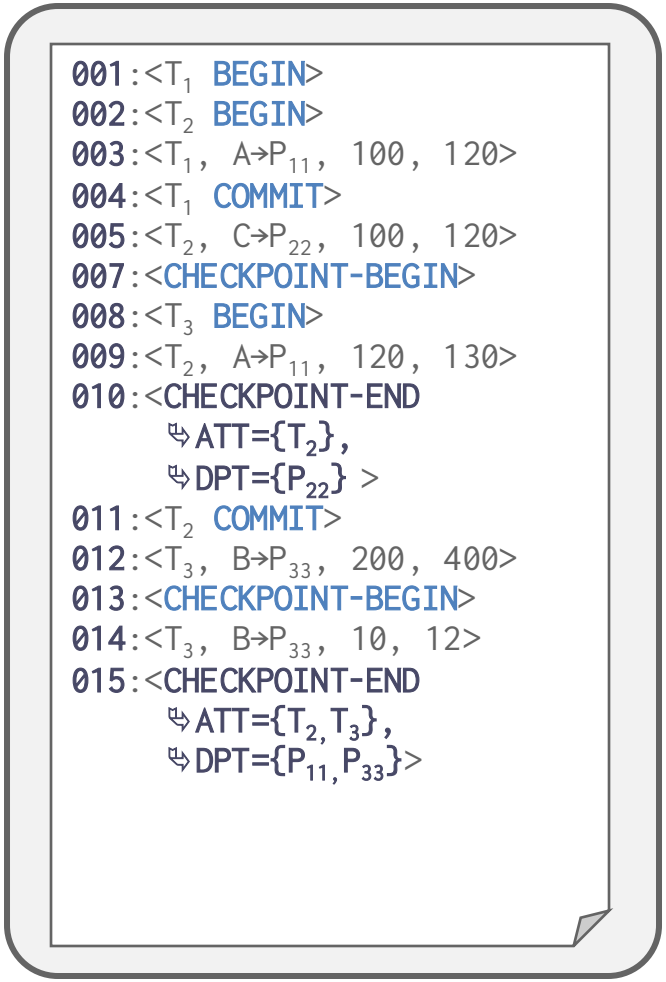
- A **fuzzy checkpoint** is where the DBMS allows active txns to continue to run while the system writes the log records for checkpoint.
  - No forced flushing. Flushes happen asynchronously and periodically.
- New log records to track checkpoint boundaries:
  - `CHECKPOINT-BEGIN`: Indicates start of checkpoint
  - `CHECKPOINT-END`: Contains checkpoint state when the checkpoint started.

Dirty Page Table (DPT)

- Separate data structure to track which pages in the buffer pool contain changes that the DBMS has not flushed to disk yet.
- One entry per dirty page in the buffer pool:
  - `recLSN`: The LSN of the oldest log record that modified the page since the last time the DBMS wrote the page to disk. This allows the DBMS to reason about whether the page was modified before or after the checkpoint started.

Fuzzy Checkpoints

- Assume the DBMS flushes P_{11} before the first checkpoint starts.
- Any txn that begins after the checkpoint starts is excluded from the ATT in the CHECKPOINT-END record.
- The LSN of the CHECKPOINT-BEGIN record is written to the MasterRecord when it completes.

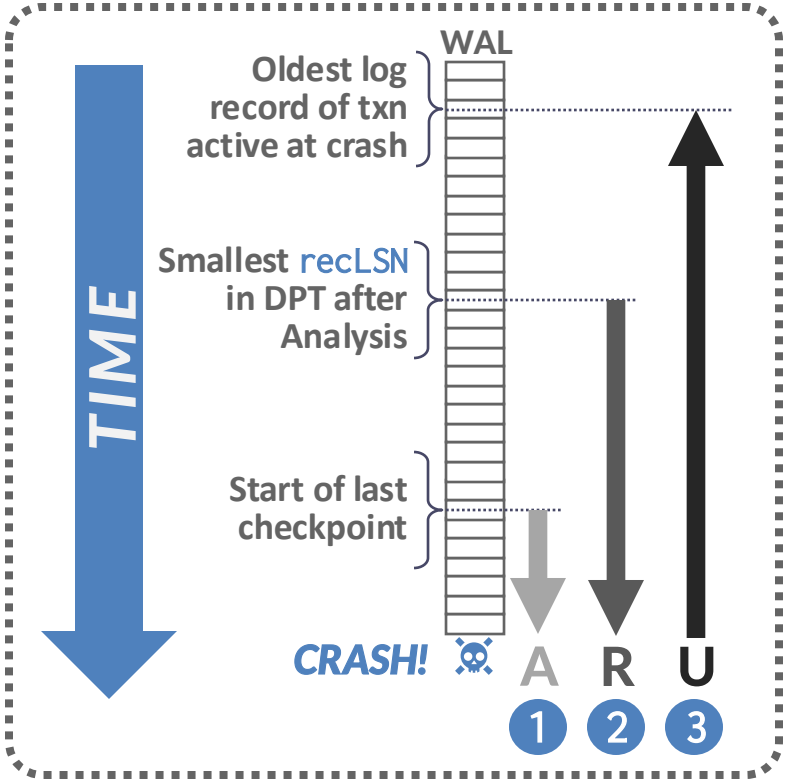


ARIES: Full Algorithm

- **Phase #1: Analysis**
  - Examine the WAL in forward direction starting at MasterRecord to identify dirty pages in the buffer pool and active txns at the time of the crash.
- **Phase #2: Redo**
  - Repeat all actions starting from an appropriate point in the log (even txns that will abort).
- **Phase #3: Undo**
  - Reverse the actions of txns that did not commit before the crash.

ARIES: Full Algorithm

- Start from last `BEGIN-CHECKPOINT` found via `MasterRecord`.
- **Analysis:** Figure out which txns committed or failed since checkpoint.
- **Redo:** Repeat all actions.
- **Undo:** Reverse effects of failed txns.



Analysis Phase

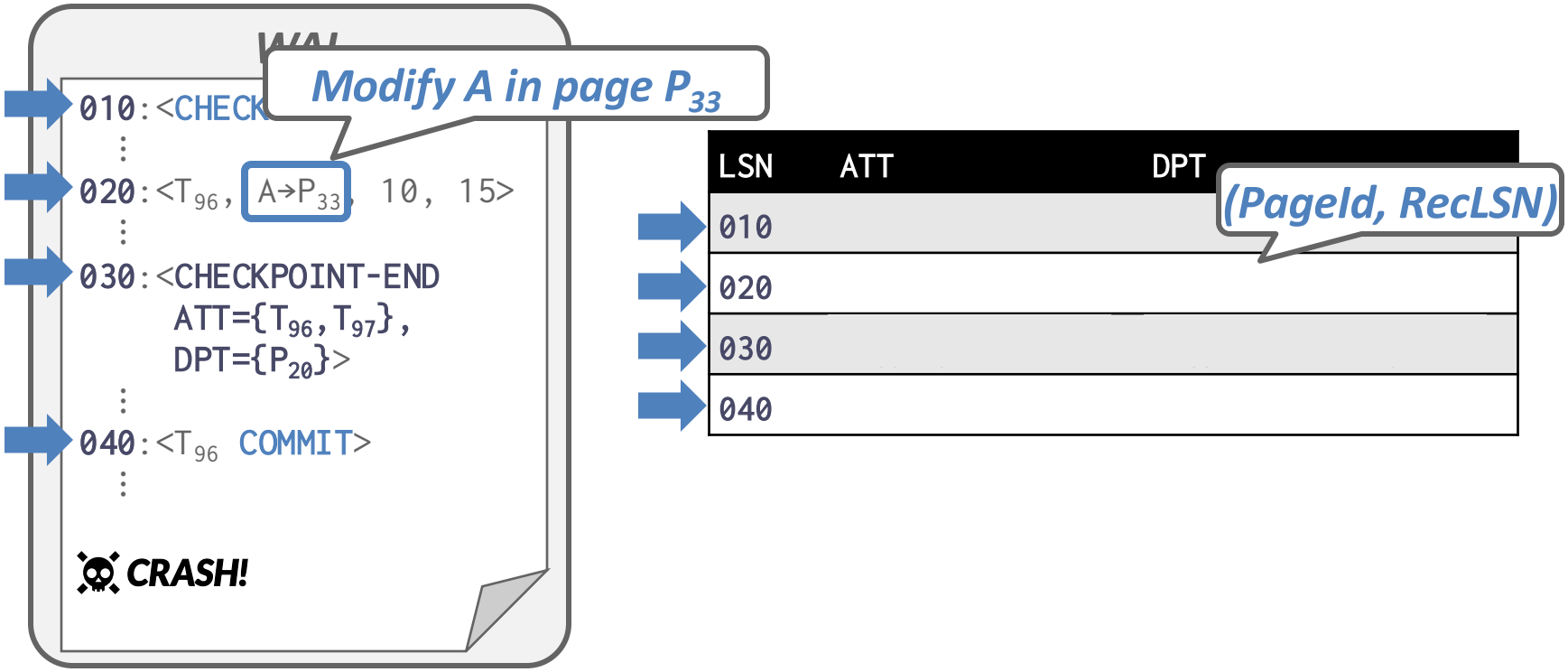
- Scan log forward from the beginning of the last successful checkpoint to the end of the log and reconstruct the **ATT**.
- If the DBMS finds a `COMMIT/ABORT` record, remove its corresponding txn from **ATT**.
- All other records:
  - If txn not in **ATT**, add it and presume we will abort it
  - On commit, remove it
- For update log records:
  - If page P_x not in **DPT**, add P_x to **DPT**, set its `recLSN`=`LSN`.

Analysis Phase

- At end of the Analysis Phase:
  - **ATT** identifies which txns were active at time of crash.
  - **DPT** identifies which dirty pages might not have made it to disk.

- These are guaranteed to be accurate because we replayed the “fuzzy” region between checkpoint start and end

Example


WAL
010:<CHECK
020:<T96, A→P33, 10, 15>
030:<CHECKPOINT-END
ATT={T96,T97},
DPT={P20}>
040:<T96 COMMIT>
CRASH!
Modify A in page P33
LSN
ATT
DPT
(PageId, RecLSN)
010
020
030
040

Redo Phase

- The goal is to reconstruct the database state at the moment of the crash:
  - Reapply all updates (even aborted txns!) and redo CLRs.
- Recall: this is necessary for (physio)logical undos to be correct

Redo Phase

- Scan forward from the log record containing smallest recLSN in **DPT**.
- For each update log record or CLR with a LSN:
  - Skip if target page is not in **DPT**.
  - Skip if the log record's LSN is less than the page's recLSN. DBMS does not need to retrieve page from disk for this check.
  - Skip if target page is in **DPT**, but that log record's LSN ≤ pageLSN. DBMS must retrieve the page from disk for this check.
  - Otherwise, apply redo

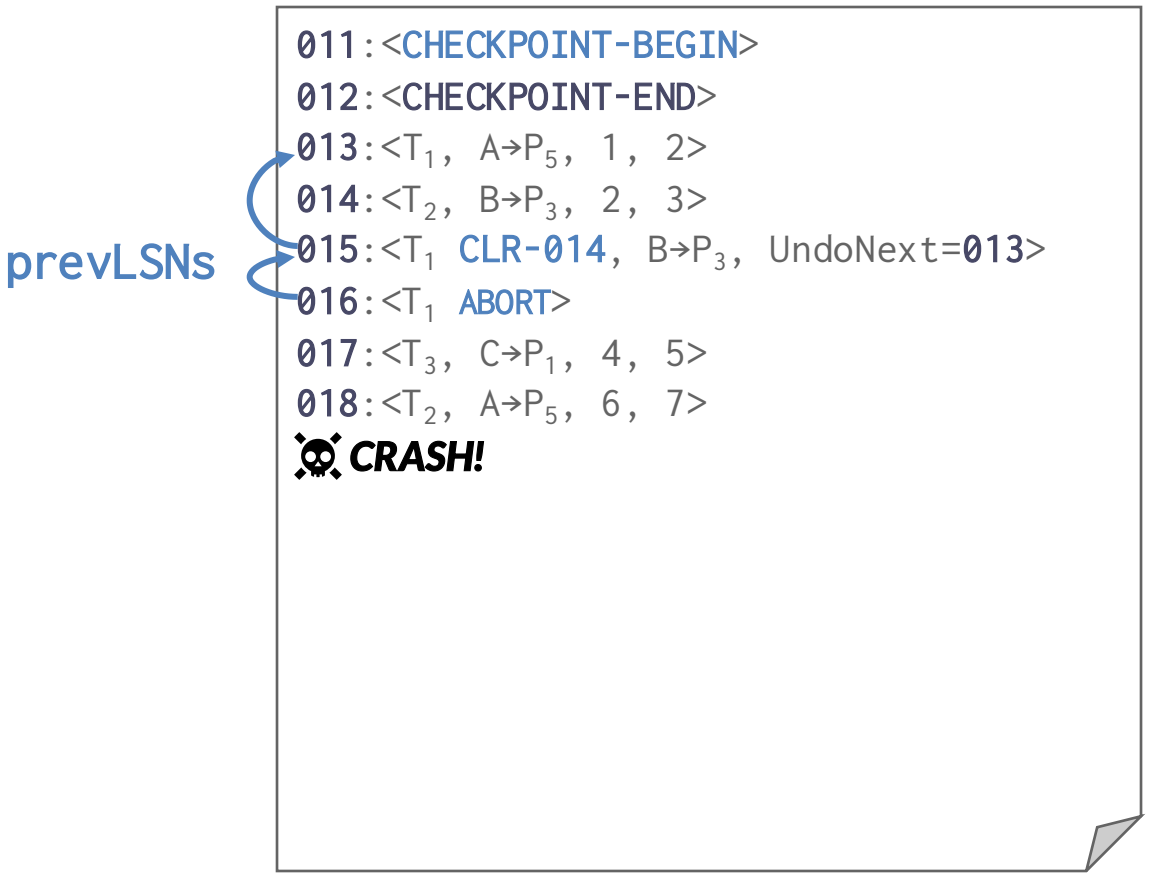
Redo Phase

- To redo an action:
  - Reapply logged update.
  - Set pageLSN to log record's LSN.
  - No additional logging, no forced flushes!

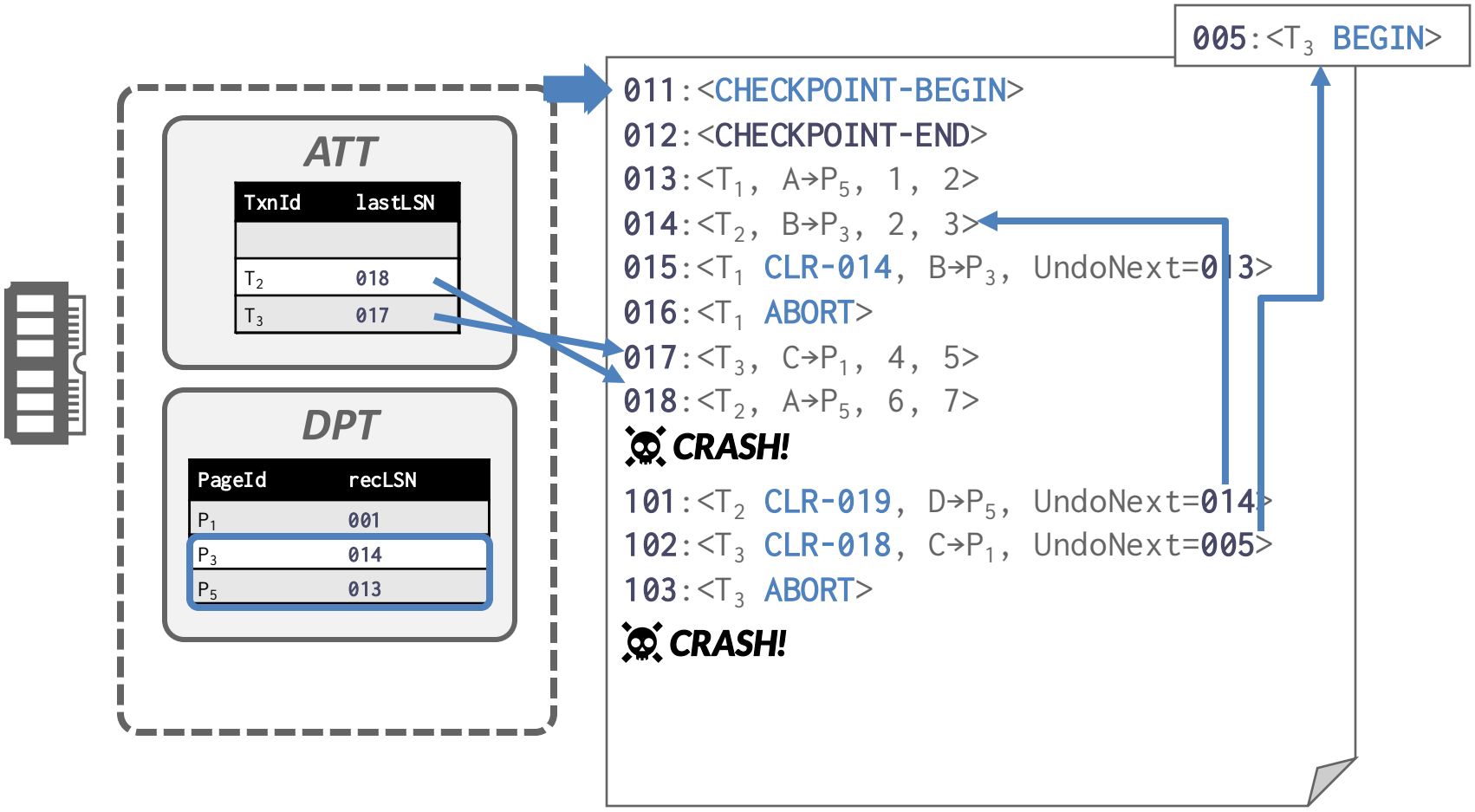
Undo Phase

- Undo all txns that were active at the time of crash and therefore will never commit.
  - These are all the txns left over in the **ATT**
- Process them in reverse LSN order using the `lastLSN` to speed up traversal.
  - At each step, pick the largest `lastLSN` across all transactions in the **ATT**.
  - Traverse `lastLSN`s in the same order, but in reverse, for how the updates happened originally.
- Write a **CLR** for every modification.

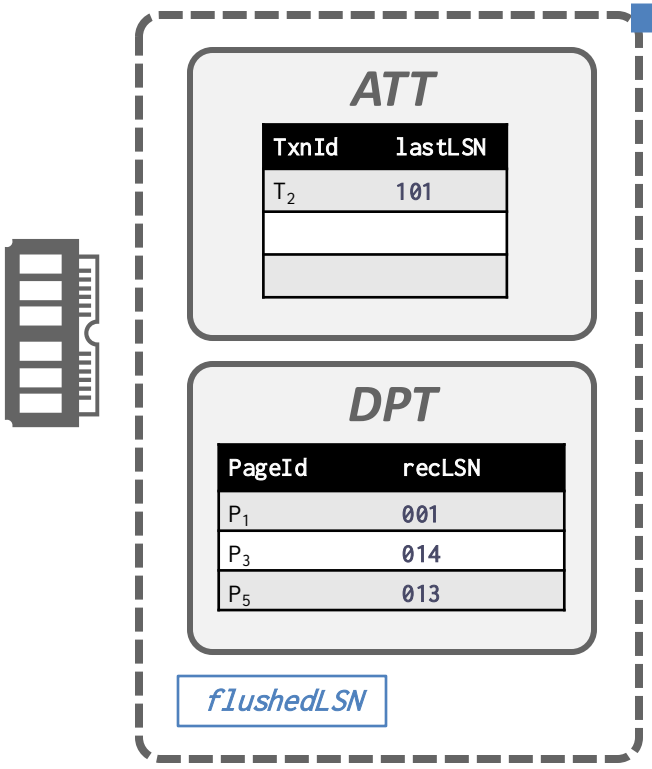
Full Example



Full Example



Full Example



011:<CHECKPOINT-BEGIN>
012:<CHECKPOINT-END>
013:<T_1, A→P_5, 1, 2>
014:<T_2, B→P_3, 2, 3>
015:<T_1 CLR-014, B→P_3, UndoNext=013>
016:<T_1 ABORT>
017:<T_3, C→P_1, 4, 5>
018:<T_2, A→P_5, 6, 7>
CRASH!
101:<T_2 CLR-019, D→P_5, UndoNext=014>
102:<T_3 CLR-018, C→P_1, UndoNext=005>
103:<T_3 ABORT>
CRASH!
201:<T_2 CLR-014, B→P_3, UndoNext=004>
202:<T_2 ABORT>

ARIES Recap

- Buffer Pool Policy: **STEAL** + **NO-FORCE**
- Write ahead logging
  - Log before page flush
  - Record LSN on page
- Fuzzy Checkpointing
  - Record ATT & DPT
  - Flush all dirty pages periodically
- Recovery: Analysis + Redo + Undo
  - Undo must generate CLRs

ARIES in GoDB

- GoDB implements most of ARIES faithfully but there are key differences
  - GoDB buffers log records in transaction context
  - Result: in-memory abort, no prevLSN, etc.
  - On recovery, active transaction load records into context during redo
  - Undo phase exercises in-memory abort

Are we done?

- What if:
  - Disk on machine fails
  - Computer won't restart
  - Missile hits your data center
  - …

- Solution: replication

Replication

- Typical approach: dedicated “hot standby”
  - Kept up to date via “log shipping” – it executes operations in the log in identical order to the primary
- May have several replicas, one nearby in local data center, one further away
  - “Half-width of a hurricane”
- Replicas often used for read-only queries
  - Have excess capacity because they are not processing xactions, just replaying log

Next Class

- Replication means our DBMS is going beyond a single machine into distributed systems territory
- This concludes our discussion of classical disk-based single-node DBMS
- Next class: Distributed Database Systems